

進階程式設計

Advanced Program Design

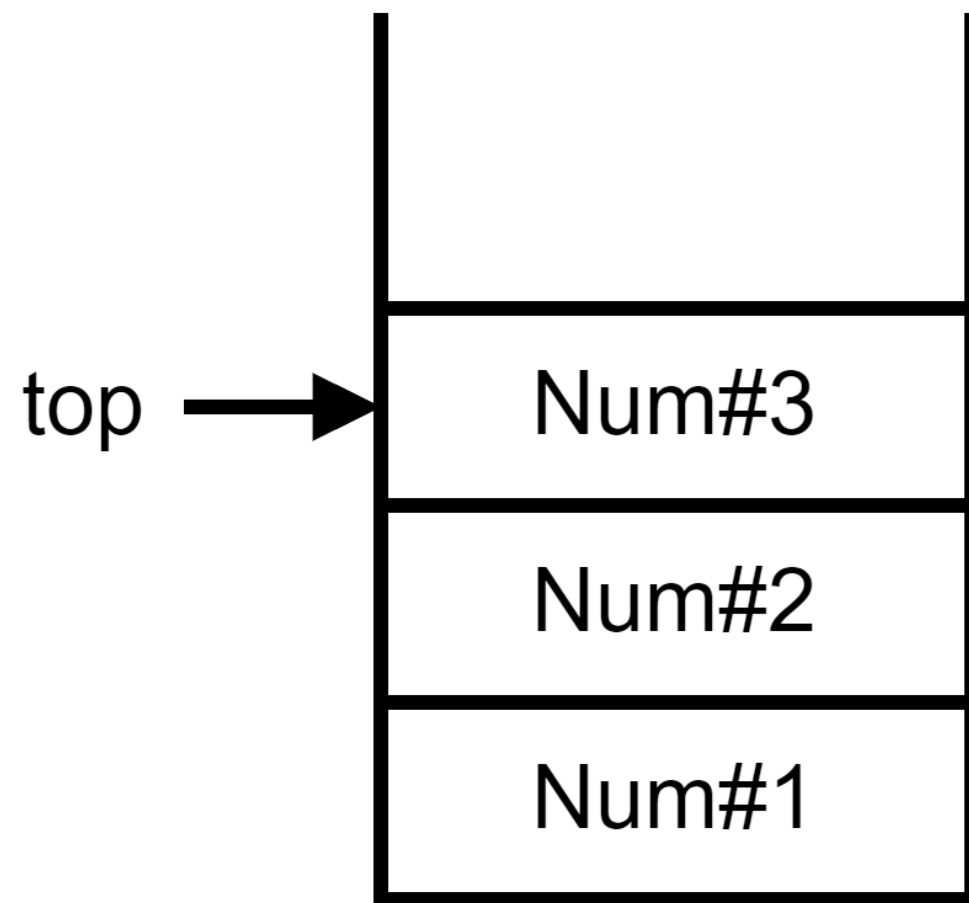
L8-1



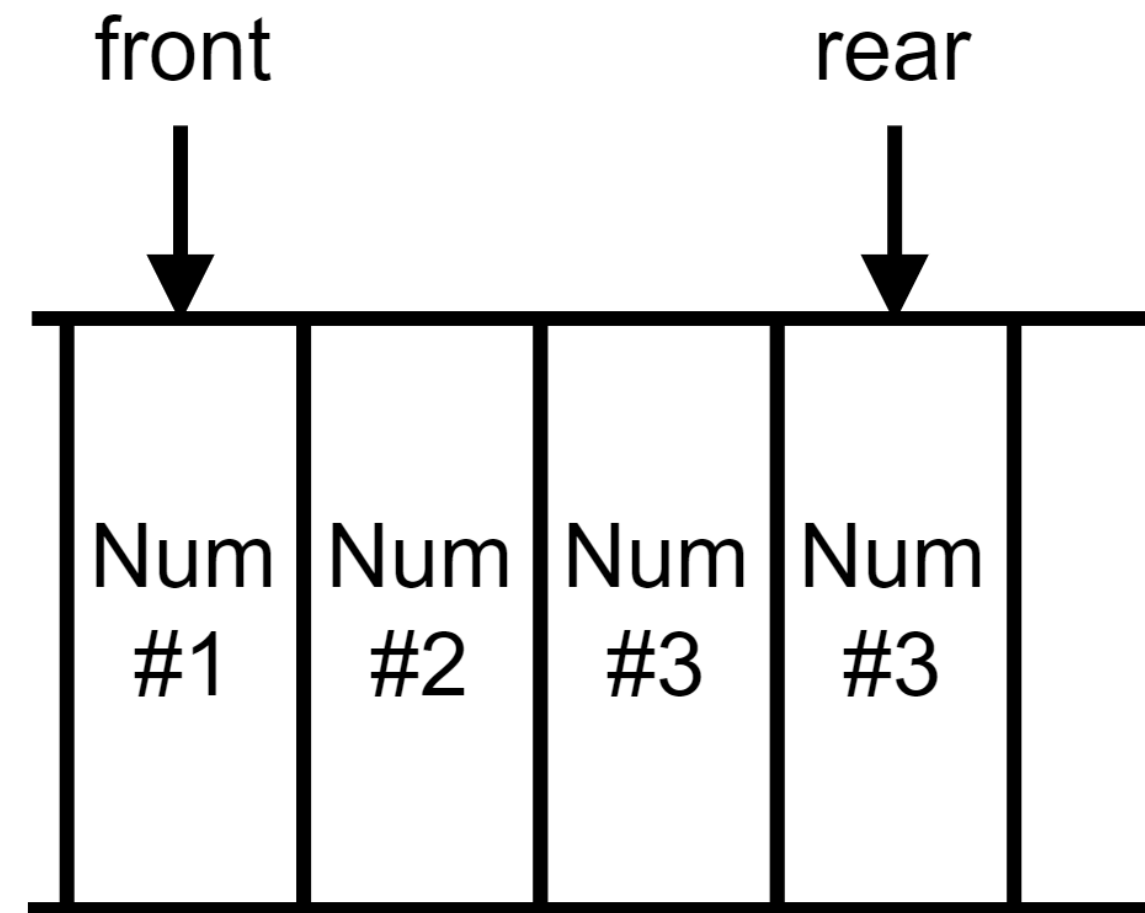
Chia-Chi Chang
ccchang@cc.ncue.edu.tw

堆疊(Stack)和佇列(Queue)

- 在演算法(Algorithm)中，堆疊(Stack)與佇列(Queue)是常用的資料結構
- 堆疊(Stack)屬於後進先出(Last In First Out, LIFO)線性串列，而且只能在頂端(top)進行資料的增加(Push)和刪除(Pop)。
- 佇列(Queue)屬於先進先出(First In First Out, FIFO)線性串列，新增資料發生在後端(rear)，而刪除資料在前端(front)。

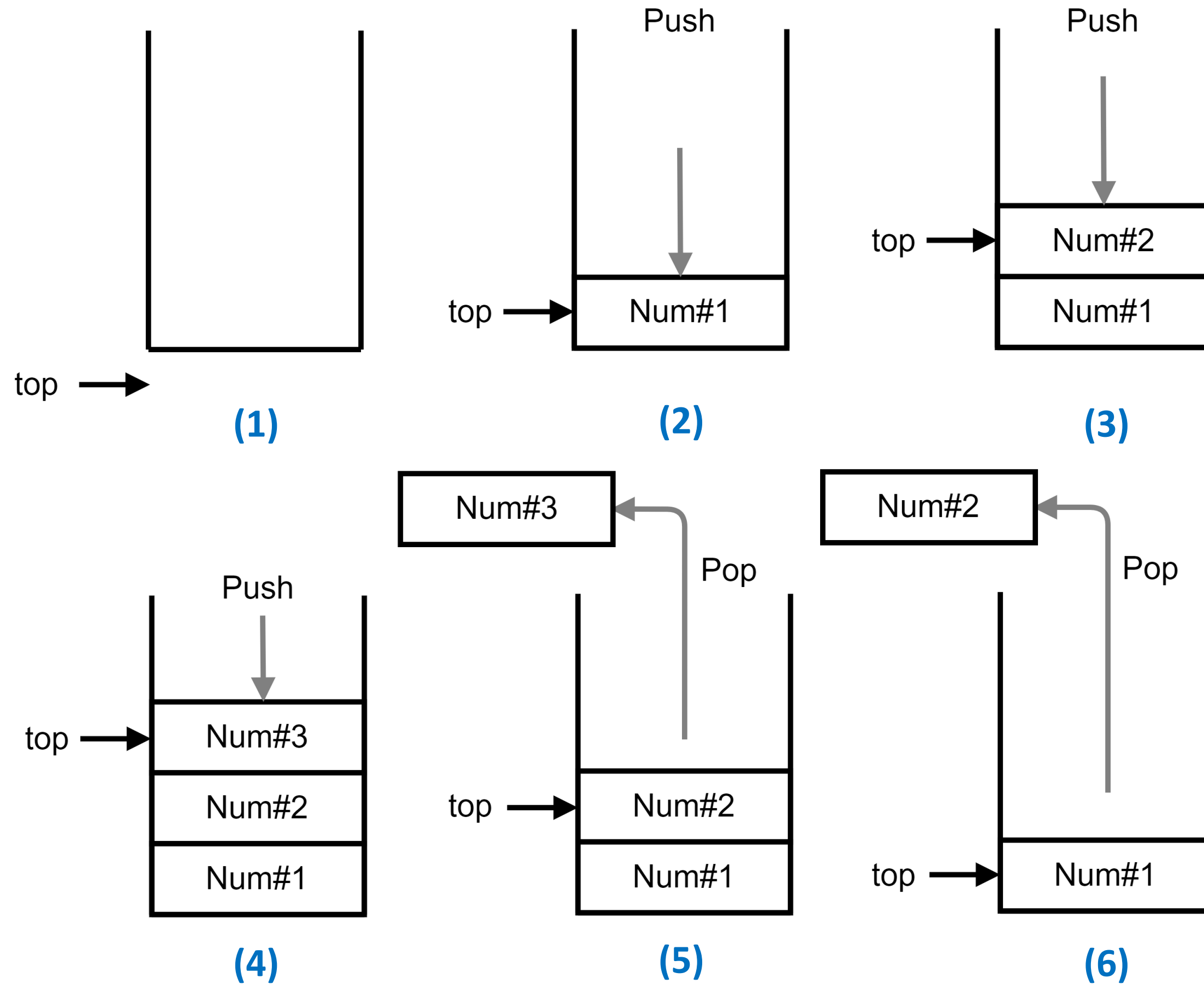


堆疊(Stack)



佇列(Queue)

堆疊(Stack)



```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#define STACK_LENGTH    100
typedef struct {
    int16_t top;
    int16_t arr[STACK_LENGTH];
} str_stack;
```

結構體實現堆疊陣列和top

```
int16_t isEmpty(str_stack* stack) {
    return (stack->top == -1) ? 1 : 0;
}
int16_t isFull(str_stack* stack) {
    return (stack->top == STACK_LENGTH-1) ? 1 : 0;
}
int16_t push(str_stack* stack, int16_t pushData) {
    if(isFull(stack)) {
        return 0;
    } else {
        stack->arr[++stack->top] = pushData;
        return 1;
    }
}
int16_t pop(str_stack* stack, int16_t* popData) {
    if(isEmpty(stack)) {
        return 0;
    } else {
        *popData = stack->arr[stack->top--];
        return 1;
    }
}
```

isEmpty, isFull, push和pop函式

```
int main(int argc, char *argv[]) {
    int16_t loop, m, cmd, num;
    str_stack stack = {-1, {0}};

    printf("Please enter the command(1:Push,2:Pop):");
    while(scanf("%hd", &cmd) != EOF) {
        switch(cmd) {
            case 1:
                printf("Enter an integer:");
                scanf("%hd", &m);
                if(push(&stack, m)) {
                    printf("The integer %hd is pushed into the stack!\n", m);
                } else {
                    printf("The stack is full!\n");
                }
                break;
            case 2:
                if(pop(&stack, &num)) {
                    printf("The popped value is %hd\n", num);
                } else {
                    printf("The stack is empty!\n");
                }
                break;
            default:
                printf("Your command is error!\n");
                break;
        }
        printf("Please enter the command(1:Push,2:Pop):");
    }
    return 0;
}
```

```
Please enter the command(1:Push,2:Pop):2
The stack is empty!
Please enter the command(1:Push,2:Pop):1
Enter an integer:50
The integer 50 is pushed into the stack!
Please enter the command(1:Push,2:Pop):1
Enter an integer:60
The integer 60 is pushed into the stack!
Please enter the command(1:Push,2:Pop):7
Your command is error!
Please enter the command(1:Push,2:Pop):1
Enter an integer:70
The integer 70 is pushed into the stack!
Please enter the command(1:Push,2:Pop):2
The popped value is 70
Please enter the command(1:Push,2:Pop):2
The popped value is 60
Please enter the command(1:Push,2:Pop):2
The popped value is 50
Please enter the command(1:Push,2:Pop):2
The stack is empty!
Please enter the command(1:Push,2:Pop):^Z
```

```
-----
Process exited after 33.61 seconds with return value 0
Press any key to continue . . .
```


中置表示法(Infix Notation)

- 人類習慣用中置表示法去描述運算式，亦即將運算子放在運算元中間(Humans generally write expressions like $3 - 4$ and $7 * 9$ in which the operator (- or * here) is written between its operands, called infix notation.)
- 中置表示法：
 - $(6 + 2) * 5 - 8 / 4$
 - $1 + 2 * 3 - 4 / 2$ is different from $((1 + 2) * 3) - (4 / 2)$
- 編譯器用堆疊去計算運算式和產生機械碼(Stacks are used by compilers to help in the process of evaluating expressions and generating machine language code.)

後置表示法(Postfix Notation)

- 波蘭數學家盧卡西維茨於1920引入數學表示式，又稱為逆向波蘭表示法(Reverse Polish Notation, RPN)，後置表示法不需要用括號來識別運算子的優先級。
- 後置表示法利用堆疊，可以節省電腦主記憶體訪問。
- 電腦傾向後置表示法有利於運算式計算，亦即把運算子放在運算元的右邊(Computers prefer postfix notation in which the operator is written to the right of its two operands.)

中置表示法	後置表示法
$(6 + 2) * 5 - 8 / 4$	$6\ 2\ +\ 5\ *\ 8\ 4\ /\ -$
$1 + 2 * 3 - 4 / 2$	$1\ 2\ 3\ *\ +\ 4\ 2\ /\ -$
$(((1 + 2) * 3) - (4 / 2))$	$1\ 2\ +\ 3\ *\ 4\ 2\ /\ -$

中置表示法轉換成後置表示法－加括號去除法

人類使用的方法，主要用於考試，快速得到後序表示法

1. 括號先行處理
2. 由內而外，由左至右，將括號去除
3. 2個運算元+1個運算子為一組，依序排列運算元，將運算子放在運算元的右邊

$$(6 + 2) * 5 - 8 / 4$$



$$\underline{6\ 2\ +} * 5 - 8 / 4$$



$$\underline{6\ 2\ +\ 5\ *} - 8 / 4$$



$$\underline{6\ 2\ +\ 5\ *} - \underline{8\ 4\ /}$$



$$6\ 2\ +\ 5\ * \ 8\ 4\ / \ -$$

$$1 + 2 * 3 - 4 / 2$$



$$1 + \underline{2\ 3\ *} - \underline{4\ 2\ /}$$



$$\underline{1\ 2\ 3\ * \ +} - \underline{4\ 2\ /}$$



$$1\ 2\ 3\ * \ + \ 4\ 2\ / \ -$$

$$(((1 + 2) * 3) - (4 / 2))$$



$$((\underline{1\ 2\ +} * 3) - \underline{4\ 2\ /})$$



$$(\underline{1\ 2\ +\ 3\ *} - \underline{4\ 2\ /})$$



$$1\ 2\ +\ 3\ * \ 4\ 2\ / \ -$$

中置表示法轉換成後置表示法－演算法

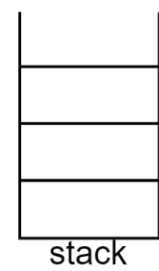
- 先將中置運算式左右兩端分別放置'('和')'，並依序儲存至字元陣列infix
 - ※若讀取中置運算式為 $(6+2)*5-8/4$ ，先將其變成 $((6+2)*5-8/4)$ 後存入字元陣列infix
- 產生的後置運算式儲存至字元陣列postfix
- 堆疊區用來存放運算子

=====

- 1) 依序由左至右讀取infix陣列裡的每1個字元，然後進行下列步驟
 - 1) 若字元為'('就加入至堆疊區。
 - 2) 若字元為')'就將堆疊區的運算子取出並加入至postfix陣列，直到遇到'('在堆疊區頂端，最後取出該'('並丟棄。
 - 3) 若字元為數字則複製到postfix陣列的下一個元素。
 - 4) 若字元為運算子，若堆疊區頂端的運算子(若有的話或不只一個)的優先權 $\geq$ 目前運算子，則取出堆疊區的運算子並且依序插入至postfix陣列的下一個元素，最後把目前運算子加入至堆疊區。

$$(6+2) * 5 - 8 / 4$$

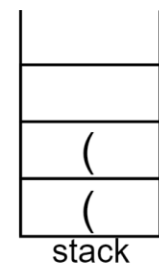
STEP 0 $(6 + 2) * 5 - 8 / 4 \longrightarrow ((6 + 2) * 5 - 8 / 4)$



infix ((6 + 2) * 5 - 8 / 4)

postfix [][][][][][][][][]

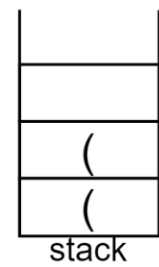
STEP 1



infix ((6 + 2) * 5 - 8 / 4)

postfix [][][][][][][][][]

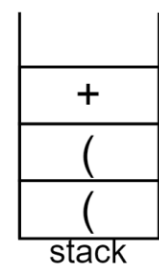
STEP 2



infix ((6 + 2) * 5 - 8 / 4)

postfix 6 [][][][][][][][]

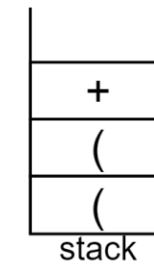
STEP 3



infix ((6 + 2) * 5 - 8 / 4)

postfix 6 [][][][][][][][]

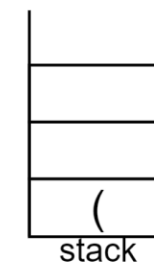
STEP 4



infix ((6 + 2) * 5 - 8 / 4)

postfix 6 2 [][][][][][][]

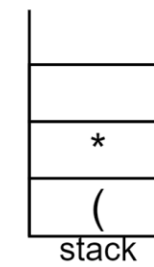
STEP 5



infix ((6 + 2) * 5 - 8 / 4)

postfix 6 2 + [][][][][][][]

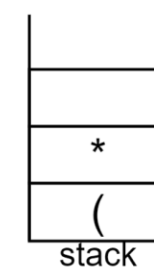
STEP 6



infix ((6 + 2) * 5 - 8 / 4)

postfix 6 2 + [][][][][][][]

STEP 7

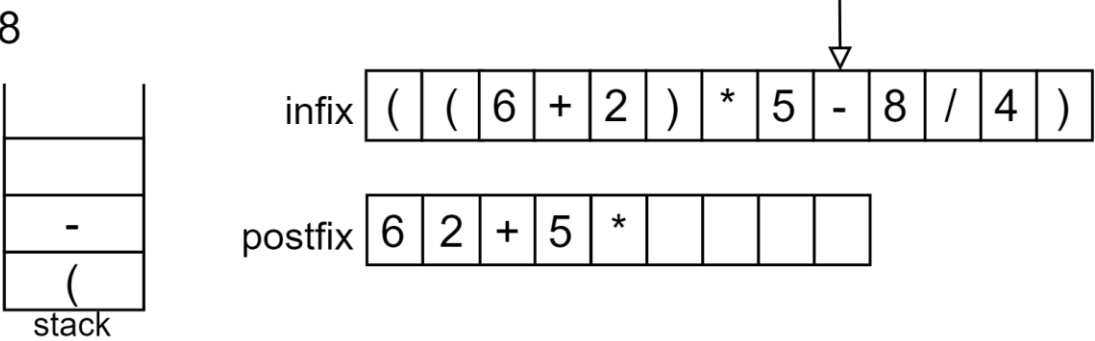


infix ((6 + 2) * 5 - 8 / 4)

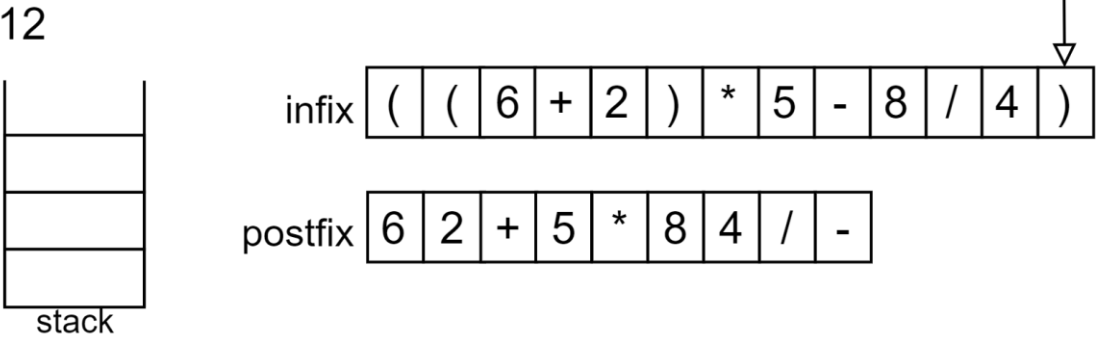
postfix 6 2 + 5 [][][][][][][]

(6+2) * 5 - 8 / 4

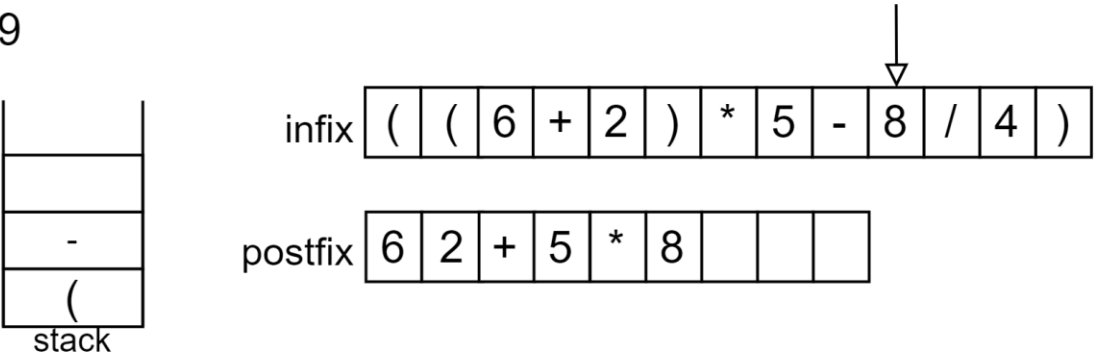
STEP 8



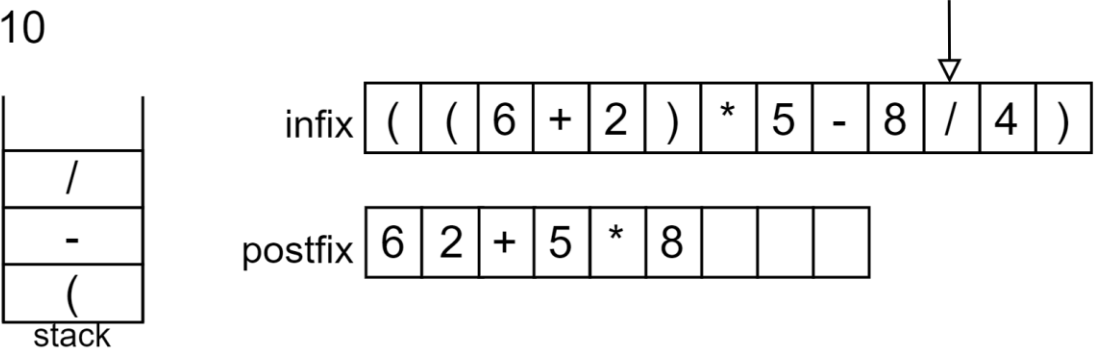
STEP 12



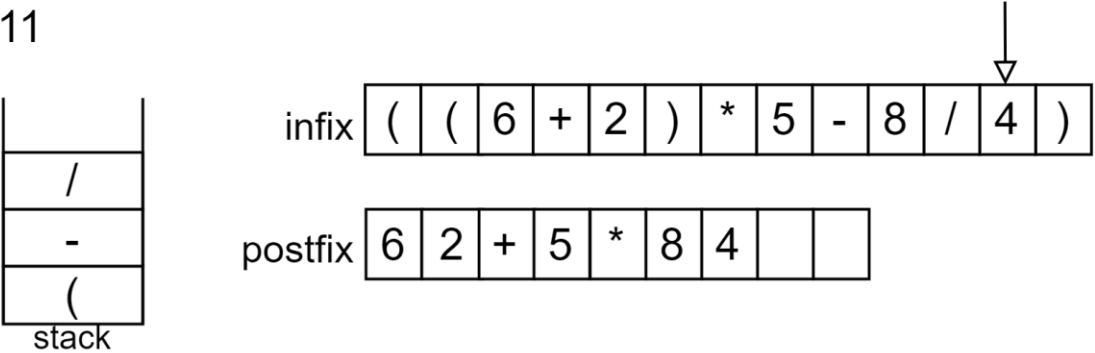
STEP 9



STEP 10



STEP 11



The End

